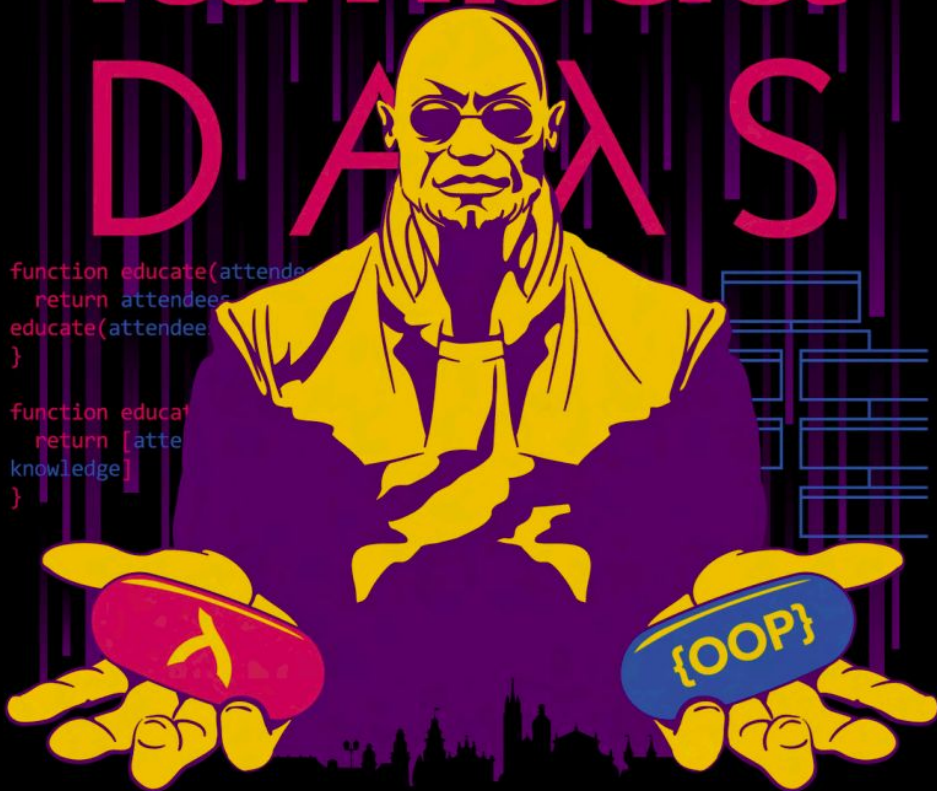




lambda D A λ S



```
function educate(attendee)
  return attendee
educate(attendee)
```

```
function educate
  return [atte
    knowledge]
```

Andor
Penzes

lambda
D A λ S
27-28 MAY 2024
KRAKÓW | POLAND



lambda D A λ S

27-28 MAY 2024

KRAKÓW | POLAND

Imagine a dependently typed Python

Andor Penzes
jww Thorsten Altenkirch



lambda
D A S
27-28 MAY 2024
KRAKÓW | POLAND



Did you just take both pills?

imgflip.com

Project status

- Design
- Prototyping
- Discussions
- Call for contributions

github / andorp / DepPy

Dependent Types

- Types and values live in the same space
- Values are part of types
- Types can be inspected like values
- Computation is done at type-checking

[1] typing.readthedocs.io/en/latest/spec (Python)

[3] <https://cseweb.ucsd.edu/~rchugh/research/nested/djs.pdf> (JS)

[4] <https://www.cs.nott.ac.uk/~psztja/publ/lics01.pdf> (Normalization)

Foundations

- We write programs to solve problems
- We organize information as data types
- We transform values of data types via functions
- Sometimes abstractions are lousy and need external help
- Sometimes we want to change the programs

Data representation

- Sum of products, generics, levitation
- Object Oriented Programming (Python)
 - Subclasses are the sums
 - Objects are the products
- FP (Haskell)
 - Data constructors are the sums
 - Fields are the products
- Python
 - Everything is a dict
 - Tagged union of everything

[1] typing.readthedocs.io/en/latest/spec (Python)

[3] <https://cseweb.ucsd.edu/~rchugh/research/nested/djs.pdf> (JS)

[6] <https://personal.cis.strath.ac.uk/conor.mcbride/levitation.pdf> (Levitation)

Compile time / Execution time?

- Compile time types ensure consistencies
- Runtime types
 - define representation
 - different interpretations of the same
- Executed tests ensures consistencies
- Assertions ensures consistencies

Nat and Fin

```
class Nat:
```

```
class Zero(Nat):
```

```
    def __init__(self):  
        pass
```

```
class Succ(Nat):
```

```
    def __init__(self, n:Nat):  
        self.n = n
```

```
class Fin:  
    # n : Nat
```

```
class FZ(Fin):
```

```
    def __init__(self, n:Nat):  
        self.n = n
```

```
class FS(Fin):
```

```
    def __init__(self, f:Fin):  
        self.n = Succ(f.n)  
        self.f = f
```

Vect

```
class Vect:
    # n : Nat
    def append(self,ys:Vect)
        -> Vect [n = self.n + ys.n]:
        pass

class Nil(Vect):
    def __init__(self):
        self.n = Zero()

    def append(self,ys:Vect):
        # ys
        # : Vect [n = ys.n]
        # : Vect [n = 0 + ys.n]
        # : Vect [n = self.n + ys.n]
        return ys
```

```
class Cons(Vect):

    def __init__(self,x,xs:Vect):
        self.n = Succ(xs.n)
        self.x = x
        self.xs = xs

    def append(self,ys:Vect):
        zs = self.xs.append(ys)
        # zs : Vect [n = self.xs.n + ys.n]
        ws = Cons(self.x,zs)
        # ws
        # : Vect [n = Succ (self.xs.n + ys.n)]
        # : Vect [n = Succ (self.xs.n) + ys.n]
        # : Vect [n = self.n + ys.n]
        return ws
```

Type checking by normalization

- Classes, objects and expressions
- Evaluation of closed expressions lead to objects
- Evaluation of open expressions lead to objects with partially applied expressions
- Intermediate form of expressions are good for debugging

[3] <https://cseweb.ucsd.edu/~rchugh/research/nested/djs.pdf> (JS)

[4] <https://www.cs.nott.ac.uk/~psztta/publ/lics01.pdf> (Normalization)

Type System

class
inst-vars
methods
init
super class
expressions
types
variable
method-app
dot
lambdas
class names
[contr]

$(x_0:A_0, \dots, x_n:A_n) \rightarrow B$

contr
init-var = e
 $i: \text{Fin} \quad i.n = 0$
 $\hline i: \text{Fin}[n=0]$

$i. \text{lookup Nil}$
 $\text{Fin}. \text{lookup Nil} : (i: \text{Fin}[n=0]) \rightarrow \text{Any}$
 $\text{Fin}[n=0] \subseteq \text{Fin}$

class context Σ

class-name $\rightarrow \text{type}$

m-var $\rightarrow \text{m-type}$

init $\rightarrow (\text{ivar} : \text{type})^*$

$\Pi = x_0:A_0 \dots x_n:A_n$

$\Sigma, \Pi \vdash x : \Pi(x)$

$\Sigma, \Pi \vdash m : (\Delta)^B \quad \Sigma, \Pi \vdash \vec{\alpha} : \Delta$

$\Sigma, \Pi \vdash m(\vec{\alpha}) : B[\vec{\alpha}]$

$\Sigma \vdash C : \text{Class} \quad \Sigma, \Pi \vdash a : C$

$\Sigma, \Pi \vdash a.i : \text{ivar-type}(C, i)$

$\Sigma \vdash C : \text{Class} \quad \Sigma, \Pi \vdash a : C \quad i \in \text{ivar}(C) \quad m \in \text{method}(C)$

$\Sigma, \Pi \vdash a.m : \text{mvar-type}(C, m) \ \a

$\Pi \vdash 0$
 $\text{m}(a, -) : (i: \text{Fin}[n=0]) \rightarrow \text{Any}$

$a.m(\vec{\alpha})$

$\Sigma, \Pi \vdash C : \text{Class}$

$\Sigma, \Pi \vdash \text{ivar}(C) = \Delta \quad \Sigma, \Pi \vdash \vec{\alpha} : \Delta$

$\Sigma, \Pi \vdash C(\vec{\alpha}) : C$

$C(\vec{\alpha}).i = \vec{\alpha}$

$\Sigma, \Pi \vdash a : A \quad \Sigma, \Pi \vdash A \leq B$
 $\Pi \vdash a : B$

$\Sigma \vdash C : \text{Class} \quad \Sigma, \Pi \vdash C \leq \text{super}(C)$

$\Sigma, \Pi \vdash A : \text{type} \quad \Sigma, \Pi \vdash C : \text{contr}$
 $\Sigma, \Pi \vdash A[C] \in A$

$A \leq B$
 $A[C] \leq B[C] ?$

$\text{m}(\vec{a}_1)$

$a.m(\vec{b}_0)(\vec{b}_1)$

Side effects

- Don't mention them
- Use Monads / Monad Transformers
- Built-in Effect Handlers

[1] typing.readthedocs.io/en/latest/spec (Python)

[2] www.unison-lang.org/docs/fundamentals/abilities/ (Unison)

[3] <https://cseweb.ucsd.edu/~rchugh/research/nested/djs.pdf> (JS)

[5] <https://arxiv.org/abs/1611.09259v> (Frank)



Thank you for your attention!

`github / andorp / DepPy`

LOOK INTO MY EYES



AND USE DEPENDENT TYPES

lambda
D A λ S

27-28 MAY 2024
KRAKÓW | POLAND

References

- [1] typing.readthedocs.io/en/latest/spec (Python)
- [2] www.unison-lang.org/docs/fundamentals/abilities/ (Unison)
- [3] <https://cseweb.ucsd.edu/~rchugh/research/nested/djs.pdf> (JS)
- [4] <https://www.cs.nott.ac.uk/~psztxa/publ/lics01.pdf> (Normalization)
- [5] <https://arxiv.org/abs/1611.09259v> (Frank)
- [6] <https://personal.cis.strath.ac.uk/conor.mcbride/levitation.pdf> (Levitation)